



On-Device Visual Accessibility Assistant

Technical Summary & Architecture Overview

Version 1.0 | Android (arm64-v8a) | minSdk 26 | targetSdk 34
Kotlin 2.3.0 | LiteRT-LM 0.16.1 | Gemma 3n E2B int4

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Architecture
4. Core Components
5. VLM Engine (Gemma 3n E2B)
6. Camera Pipeline
7. Speech Recognition & TTS
8. Adaptive Memory System
9. Performance Optimizations
10. Safety & Stability
11. Tech Stack
12. File Structure

1. Executive Summary

Vyze is an offline, on-device visual accessibility assistant for Android that helps blind and visually impaired users understand their surroundings in real time. The app uses a local Gemma 3n E2B (Edge 2 Billion) multimodal vision-language model running entirely on-device via Google's LiteRT-LM GPU backend to describe camera scenes, read text, identify objects, and provide spatial navigation cues -- all without any internet connection or cloud dependency.

Users interact hands-free through continuous speech recognition: they speak a question like "What is in front of me?" or "Read this label", and Vyze captures a camera frame, runs VLM inference, and speaks the result aloud via text-to-speech. The app supports a continuous "point-and-describe" scanning mode similar to Gemini Live.

Key differentiators: (1) 100% offline -- no data leaves the device, (2) real-time voice-first UX with barge-in interruption, (3) adaptive memory that learns from past interactions, (4) sub-second time-to-first-token via streaming sentence-buffered TTS, and (5) aggressive latency optimizations including 256px image clamping, greedy decoding, and GPU kernel pre-warming.

2. Problem Statement

Over 2.2 billion people globally have vision impairments (WHO, 2023). Existing assistive technology relies heavily on cloud-based AI services that require internet connectivity, introduce latency (1-3 seconds per query), raise privacy concerns by uploading camera images, and fail in offline environments (rural areas, subways, international travel).

Vyze addresses these gaps by running the entire vision-language pipeline locally on the user's phone. The Gemma 3n E2B model (3.66 GB int4 quantized) fits within mobile GPU memory constraints while providing high-quality scene understanding and text extraction capabilities.

3. Solution Architecture

Vyze follows a modular architecture with clear separation of concerns:

- Camera Layer: CameraX ImageAnalysis delivers frames to an in-memory buffer (zero disk I/O)
- VLM Engine: LiteRT-LM + Gemma 3n E2B performs multimodal inference on GPU
- Prompt Builder: Dynamic prompt system with navigation, direct-query, and continuous-mode rules
- TTS Manager: Neural voice synthesis with sentence-buffered streaming for instant audio
- Speech Recognizer: Android SpeechRecognizer with aggressive silence endpoints
- Adaptive Memory: Room database with vector similarity search for contextual recall
- Core Controller: Orchestrates the full pipeline with session isolation and state management

Data Flow

User speaks "What is this?"

- > `SpeechRecognizer.onResults()` extracts text
- > `CameraFragment.bargeInAndCapture()` stops TTS + captures frame
- > `CameraSetupDelegate.takeSnapshot()` copies fresh Bitmap from `ImageAnalysis`
- > `VyzeCoreController.triggerSnapshot()` builds prompt + starts VLM inference
- > `VlmEngineManager.analyzeImage()` sends Bitmap + prompt to Gemma 3n E2B
- > `onTokenGenerated()` streams tokens into `sentenceBuffer`
- > `flushSentenceBufferIfReady()` sends clause-level chunks to `TTSManager`
- > `TTSManager.speak()` plays audio with `pendingUtteranceIds` tracking
- > `onDone()` resets to IDLE + restarts `SpeechRecognizer` for next query

4. Core Components

4.1 VyzeCoreController

The central orchestrator that manages the full capture-to-speech pipeline. Holds references to `VlmEngineManager`, `TTSManager`, `DynamicPromptBuilder`, and `MemoryRepository`. Key responsibilities:

- Session isolation via UUID-based `activeSessionId` (prevents stale response leakage)
- Coroutine-based inference with Job cancellation on new queries
- Sentence-buffered TTS streaming with clause-boundary flushing
- Duplicate detection via `lastDescribedObject` cache with 3s debounce
- Bitmap center-cropping + downscaling for VLM input normalization
- 15-second watchdog timer to prevent indefinite ANALYZING state

4.2 VlmEngineManager

Wrapper around Google's LiteRT-LM framework that manages the Gemma 3n E2B engine lifecycle. Provides the `analyzeImage()` API for multimodal inference.

- GPU-only execution via `Backend.GPU()` with OpenCL/Vulkan acceleration
- Model resolution from `/storage/emulated/0/Download/` or `getExternalFilesDir(null)`
- Bitmap preprocessing: center-crop to 1:1 square + scale to 256x256 pixels
- Gemma turn-format prompting: `<start_of_turn>user\n...<end_of_turn>\n<start_of_turn>model\n`
- Greedy decoding: `maxTokens=35`, `temperature=0.1`, `topK=1` for fast mobile inference
- GPU warm-up: dummy 1x1 image pre-compiles OpenCL/Vulkan kernels on init
- Non-destructive interrupt: `latch.countDown()` without `engine.close()`

4.3 CameraSetupDelegate

Manages CameraX use cases (Preview + ImageAnalysis) and provides zero-disk-I/O frame extraction via in-memory Bitmap buffer.

- `ImageAnalysis.Analyzer` decodes YUV_888 to ARGB_8888 Bitmap at camera framerate
- Atomic frame swap with `frameLock` prevents recycling during snapshot extraction
- Frame counter guarantees fresh frames on voice-triggered snapshots
- `takeSnapshot()` polls for post-query frame with 300ms timeout + `PreviewView` fallback

5. VLM Engine -- Gemma 3n E2B

The vision-language model is Google's Gemma 3n E2B (Edge 2 Billion parameters), int4 quantized to 3.66 GB. It runs entirely on-device via the LiteRT-LM framework with GPU acceleration.

Model Specifications

Model:	gemma-3n-E2B-it-int4.litertlm
Size:	3.66 GB (int4 quantized)
Parameters:	~2 billion
Backend:	GPU (OpenCL/Vulkan)
Framework:	LiteRT-LM 0.16.1
Max Output Tokens:	35 (greedy)
Temperature:	0.1
Top-K:	1
Image Input:	256x256 center-cropped square
Prompt Format:	Gemma turn system (<start_of_turn>)

Inference Pipeline

1. Bitmap center-crop to 1:1 square (preserves spatial alignment)
2. Scale to 256x256 pixels (minimizes prefill time)
3. JPEG encode to byte array (avoids raw Bitmap overhead)
4. Build Gemma turn-format prompt with system directive + query
5. Create Conversation with ConversationConfig(samplerConfig)
6. Engine.sendMessageAsync(contents, callback)
7. Stream tokens via onTokenGenerated callback
8. CountdownLatch.await() blocks until onDone fires
9. Return full response string to VyzeCoreController

6. Camera Pipeline

Vyze uses CameraX with two use cases: Preview (for the user to see what they're pointing at) and ImageAnalysis (for frame extraction). The pipeline is optimized for zero-disk-I/O and minimal latency.

Frame Extraction Flow

- ImageAnalysis.Analyzer runs on a single-thread ExecutorService (30fps)
- YUV_888 -> ARGB_8888 conversion with rotation correction
- New frame swapped into AtomicReference<Bitmap?> via getAndSet()
- Old frame recycled after acquiring frameLock (prevents use-after-free)
- Frame counter incremented to signal fresh frame availability

Snapshot Capture

- takeSnapshot() records frameCounter at query time
- Polls for frameCounter > counterAtQuery (guarantees fresh frame)
- 300ms timeout with fallback to latestFrame.get() if no new frame arrives

- Deep copy via `Bitmap.copy(ARGB_8888, true)` on analysis thread
- `Bitmap.isRecycled` + `getPixel(0,0)` validation before VLM inference

7. Speech Recognition & TTS

7.1 Speech Recognition

Uses Android's SpeechRecognizer with aggressive silence endpoints for fast response to short queries like "What is this?"

- COMPLETE_SILENCE_LENGTH: 400ms (was 1500ms)
- POSSIBLY_COMPLETE_SILENCE_LENGTH: 300ms (was 1500ms)
- MINIMUM_LENGTH: 1000ms (new)
- PARTIAL_RESULTS: true (enables early intent parsing)
- Auto-restart after TTS completes (continuous hands-free loop)
- 200ms settle delay after TTS stop to prevent mic hearing barge-in tap

7.2 Text-to-Speech

Neural voice synthesis via Google TTS engine with deterministic completion tracking.

- Engine: Google TTS (com.google.android.tts) for neural voice quality
- Voice selection: Highest QUALITY_HIGH/QUALITY_VERY_HIGH voice for locale
- Pitch: 0.96f (-2%) for warmer, non-metallic tone
- Rate: 0.98f (-2%) for conversational cadence
- Audio: USAGE_ASSISTANCE_ACCESSIBILITY (non-duckable)
- Volume: KEY_PARAM_VOLUME = 1.0f (stable full volume)
- Audio Focus: AUDIOFOCUS_GAIN_TRANSIENT
- Cadence: Punctuation-aware spacing for natural pauses

7.3 Sentence-Buffered Streaming

Tokens from VLM inference are buffered and flushed at natural clause boundaries for minimal time-to-first-audio.

- First chunk: flush at ≥ 1 word or ≥ 3 characters (instant feedback)
- Subsequent chunks: flush at commas/colons (12+ chars) or sentence terminators (20+ chars)
- Final flush: force-flush all remaining text on onComplete (no minimum)
- Utterance ID tracking via pendingUtteranceIds (deterministic, no polling)
- 400ms post-drain grace period for AudioTrack hardware buffer

8. Adaptive Memory System

Vyze maintains a local Room database that records interactions, user preferences, and environment context to provide increasingly personalized assistance.

Data Schema

```
InteractionRecord:
- id: Long (auto-generated)
- timestamp: Long
- imageEmbedding: FloatArray (MediaPipe Vision Embedder)
```

- rawPrompt: String
- generatedOutput: String
- userFeedback: String (edited text or preference tags)

MemoryDao:

- getAllPreferences(): List<Preference>
- getRecentEnvironment(limit): List<EnvironmentMemory>
- findSimilar(embedding, k): List<SimilarInteraction>

Contextual Prompt Injection

- Similar past interactions retrieved via vector similarity search
- User preferences injected as section in prompt payload
- Environment context (room layouts, frequent objects) included
- All queries execute off-main-thread via Dispatchers.IO

9. Performance Optimizations

- Image Preprocessing: Center-crop to 1:1 square + scale to 256x256 (4x fewer pixels than 512x512)
- Greedy Decoding: maxTokens=35, temperature=0.1, topK=1 for fastest token generation
- Minimal Prompts: CONTINUOUS_MODE_RULES: 72 chars vs 289 chars (75% fewer prefill tokens)
- First-Chunk Flush: 1 word / 3 chars triggers first TTS audio (~200ms after first tokens)
- GPU Warm-up: Dummy 1x1 image pre-compiles OpenCL/Vulkan kernels on app launch
- Zero Disk I/O: In-memory Bitmap buffer from ImageAnalysis (no takePicture + JPEG decode)
- Frame Counter: Guarantees fresh frame on every voice query (no stale frame reuse)
- Speech Rate: 1.15x TTS playback speed reduces total utterance duration
- Barge-In: Instant TTS stop on touch/speech input (200ms settle before mic restart)
- Engine Interrupt: latch.countDown() releases blocking await without destroying engine
- Session Isolation: UUID-based activeSessionId prevents stale callback leakage
- Center-Crop: Preserves spatial alignment (left/right positioning) for VLM input

10. Safety & Stability

State Machine

```

IDLE -> (tap/speech) -> ANALYZING -> (VLM complete) -> SPEAKING -> (TTS done) -> IDLE
IDLE -> (barge-in) -> ANALYZING (cancels previous + stops TTS)
ANALYZING -> (15s timeout) -> IDLE + error speech

```

Crash Prevention

- 15-second watchdog timer forces ANALYZING -> IDLE if inference hangs
- Bitmap.isRecycled + getPixel(0,0) corruption check before VLM inference
- Session ID gating on all callbacks (onTokenGenerated, onComplete, onError)
- Coroutine Job cancellation with isActive checks at 3 points in inference
- Non-destructive interrupt: engine stays alive for next query
- FrameLock prevents bitmap recycling during snapshot extraction
- isCapturing AtomicBoolean prevents concurrent frame extractions
- 1-second debounce on trigger events prevents double-fire

Memory Management

- Bitmap.recycle() in finally blocks for both original and scaled bitmaps
- CameraSetupDelegate recycles old frames after frameLock acquisition
- Error log pruning: 7-day retention window on cold start
- Conversation lifecycle: manual close() (not use{}) to prevent SIGSEGV

11. Technology Stack

Languages & Frameworks

Language:	Kotlin 2.3.0
Build System:	Gradle 8.14.3 + Kotlin DSL
Min SDK:	26 (Android 8.0)
Target SDK:	34 (Android 14)
Architecture:	ARM64-v8a only (64-bit GPU delegates)

AI & ML

VLM Model:	Gemma 3n E2B int4 (3.66 GB)
Inference:	LiteRT-LM 0.16.1 (Google)
GPU Backend:	OpenCL/Vulkan via Backend.GPU()
Embeddings:	MediaPipe Vision Embedder

Android Libraries

Camera:	CameraX 1.3.1
Database:	Room 2.7.0
Navigation:	Navigation 2.7.7
Coroutines:	kotlinx-coroutines 1.10.1
Lifecycle:	Lifecycle 2.7.0
UI:	Material 1.11.0 + ConstraintLayout 2.1.4

Build & Release

Release Build:	R8 minification + resource shrinking
JNI Packaging:	useLegacyPackaging = true (uncompressed .so)
Model Assets:	noCompress += "litertlm" (uncompressed in APK)

12. Key File Structure

VyzeApplication.kt	App singleton, dependency injection, global error handler
MainActivity.kt	Speech recognition, TTS integration, lifecycle management
VyzeCoreController.kt	Pipeline orchestrator, session management, sentence buffering
VlmEngineManager.kt	LiteRT-LM wrapper, Gemma 3n E2B engine lifecycle
CameraSetupDelegate.kt	CameraX setup, frame buffer, bitmap extraction
CameraFragment.kt	Camera UI, voice triggers, continuous mode, state machine
DynamicPromptBuilder.kt	Prompt assembly with navigation/direct/continuous rules
TTSManager.kt	Neural TTS, voice selection, utterance tracking, audio focus
MemoryRepository.kt	Adaptive memory, vector search, interaction records
VyzeDatabase.kt	Room database, DAOs, entities